

仿真软件的使用与问题解答

软件安装

软件安装：

◆ Windows使用setup.exe安装

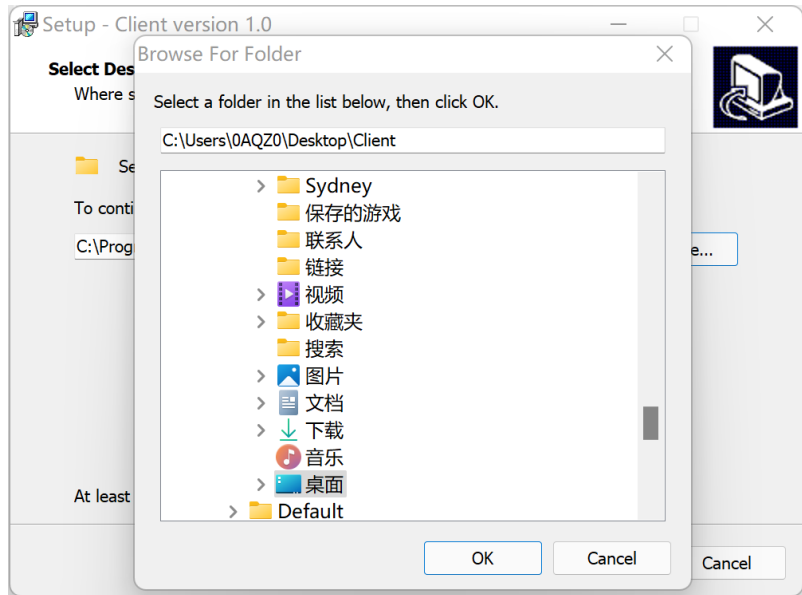
注意不要选择在需要管理员权限的目录，如

C:\Program Files (x86)\Client，否则需要**管理员权限**打开，

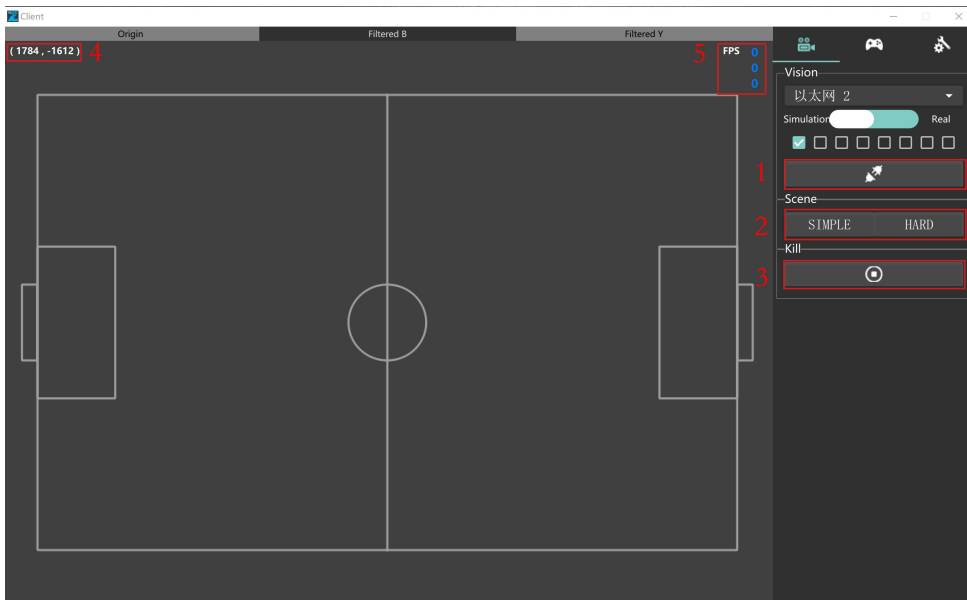
才能进行LOG录制

◆ Linux/Mac使用源代码编译

需要下载QT，并编译源码，不推荐使用

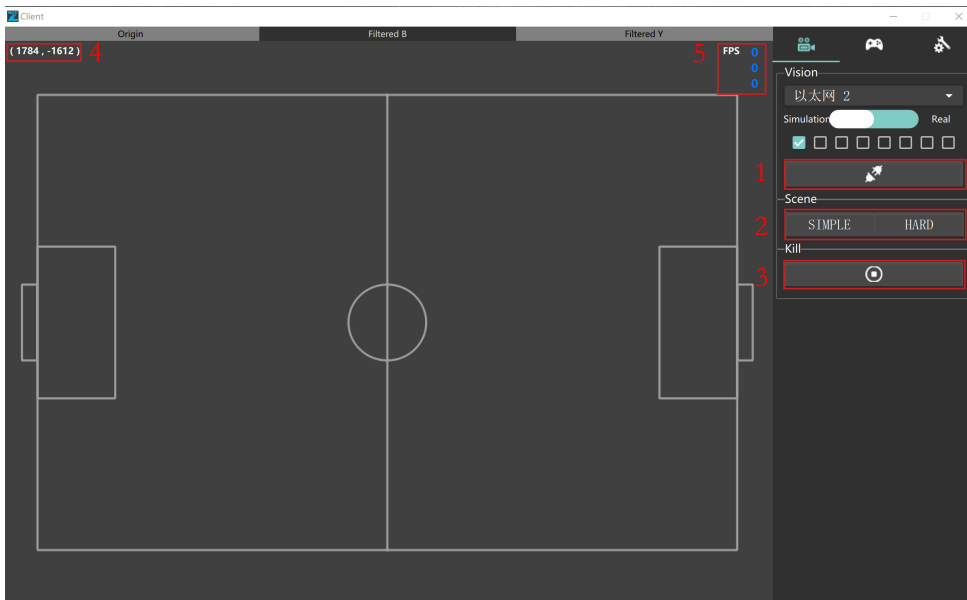


软件基本使用



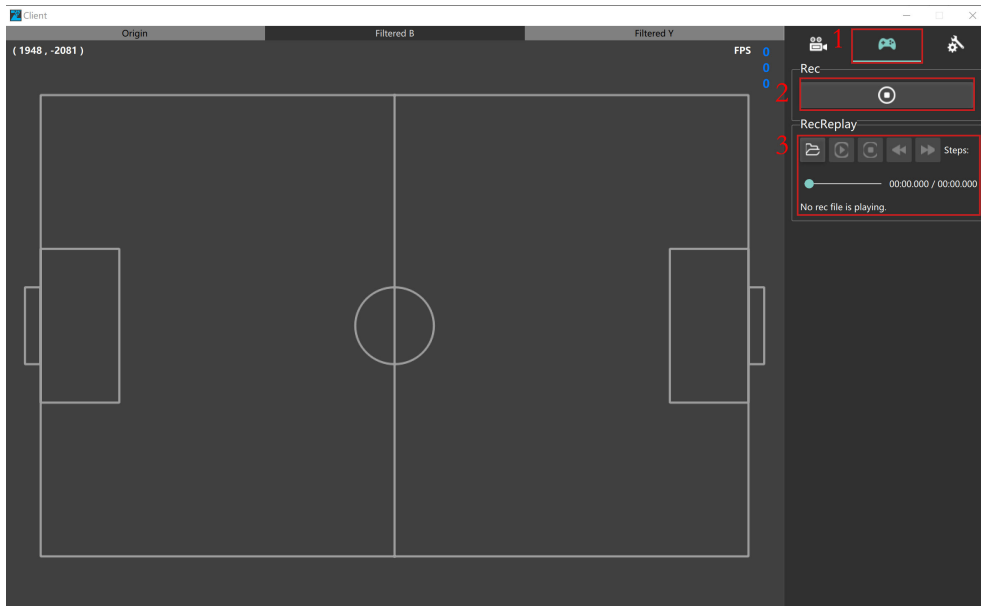
- ◆ 红色方框1可以选择连接或者断开图像
- ◆ 红色方框2选择测试的场景，simple或hard
 - simple场景：静态障碍物
 - hard场景：动态障碍物+静态障碍物
- ◆ 红色方框3可以快速关闭程序
- ◆ 红色方框4显示鼠标所指位置的坐标（mm）

软件基本使用



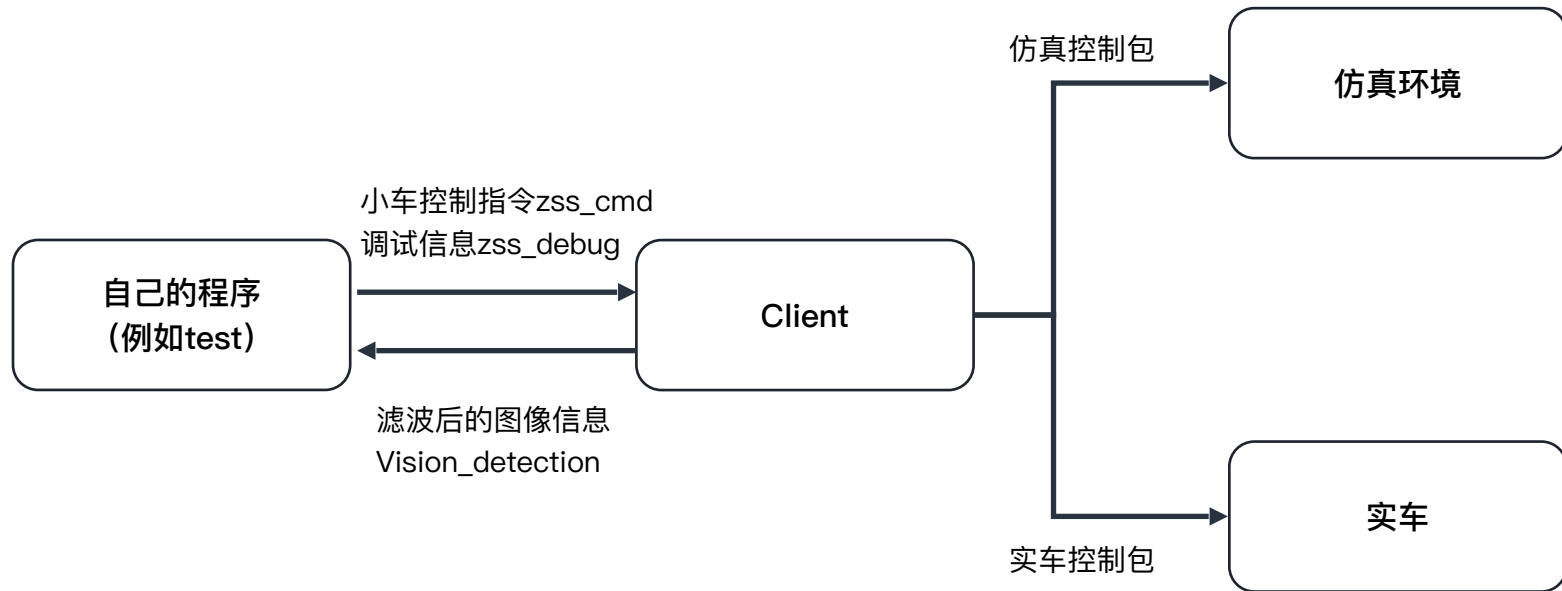
- ◆ 红色方框5分别显示图像帧数、蓝车控制指令帧数、黄车控制指令帧数
- ◆ 常用小技巧：连接图像后，使用`、1-10、-、+、i、o、p、[可以控制蓝车数量从0-16变化，如果同时按住ctrl可以控制黄车数量；鼠标可以直接拖动机器人

软件基本使用



- ◆ 红色方框1可以切换到log的工具栏
- ◆ 红色方框2开始或者停止录制log，默认开启
- ◆ 红色方框3内显示log播放信息
- ◆ 最左边文件夹按钮可以打开log文件进行播放（打开时需要断开图像连接），其他按钮分别是开始、停止、快退、快进

控制流程



Protobuf简介

- Protobuf是什么？

protobuf(Protocal Buffers)是google开发的一种用于序列化结构化数据 (JSON、XML)的一种方式。你可以定义你的数据的结构，protobuf是语言中性的，可以使用C++、C#、GO、Java、Python来读写你的数据。

- 为什么使用Protobuf？

灵活、高效，只需要关于数据结构的描述.proto，就可以使用compiler自动生成进行编码、解析的class，这个class提供了getter和setter这些method读写protobuf。而且，protobuf有更强的拓展性（读取旧的格式）。

Reference: <https://developers.google.com/protocol-buffers/docs/cpptutorial>
<https://developers.google.com/protocol-buffers/docs/pythontutorial>

Demo程序讲解

主要提供了两个Demo程序，分别完成接收图像信息、发送控制指令：

- ◆ ssl_demo_python：Python例程，需要Python>=3、Protobuf包pip install protobuf
- ◆ ssl_demo_c++：C++例程，需要QT、VS Studio进行编译

以下重点讲解Python例程：



接收图像信息

在vision.py里实现了两个类，一个是Vision用于读取图像数据，一个是Robot用于存储每个机器人的数据。

◆ Robot存储了机器人的ID、是否可见、位置、速度、朝向（分为图像滤波后和滤波前两种）

```
class Robot(object):
    def __init__(self, id, visible=False,
                 x=-999999, y=-999999, vel_x=0, vel_y=0, orientation=0,
                 raw_x=-999999, raw_y=-999999, raw_vel_x=0, raw_vel_y=0,
                 raw_orientation=0):
        self.id = id
        self.visible = visible
        # filtered vision
        self.x = x
        self.y = y
        self.vel_x = vel_x
        self.vel_y = vel_y
        self.orientation = orientation
        # raw vision
        self.raw_x = raw_x
        self.raw_y = raw_y
        self.raw_vel_x = raw_vel_x
        self.raw_vel_y = raw_vel_y
        self.raw_orientation = raw_orientation
```

接收图像信息

- ◆ Vision通过UDP读取图像数据包，并解析存储在blue_robot和yellow_robot两个数组中，用法如下：

```
from vision import Vision

# 实例化Vision这个类
vision_module = Vision()

# 读取我的机器人的位置，默认我的机器人为蓝车0号
my_robot = vision_module.my_robot
print('My robot:', my_robot.x, my_robot.y)

# 读取蓝车0号的位置，其他车号0-15类似
blue_robot_0 = vision_module.blue_robot[0]
print('Blue robot 0:', blue_robot_0.x, blue_robot_0.y)

# 读取黄车0号的位置，其他车号0-15类似
yellow_robot_0 = vision_module.yellow_robot[0]
print('Yellow robot 0:', yellow_robot_0.x, yellow_robot_0.y)
```

发送控制指令

在action.py中实现了Action这个类，用于控制我的机器人（默认是蓝车0号）的vx和vw，用法如下：注意本次实现的是**差分驱动**，不论是否发送vy都会被直接置零

```
from action import Action

# 实例化Action这个类
action_module = Action()
while True:
    # 给我的机器人发送vx为100，vw为10的速度
    action_module.sendCommand(vx=100, vw=10)
    time.sleep(0.02)
```

绘制调试信息

在Debugger中实现了Debugger这个类，用于绘制调试信息，例如画圆、画线、画点

- ◆ draw_circle——画圆，参数：圆心坐标和半径
- ◆ draw_line——画线，参数：两点的坐标 (x1,y1) 和 (x2,y2)
- ◆ draw_lines——画多条线，参数：两点的坐标列表，x1,y1,x2,y2都是列表
- ◆ draw_point——画点，参数：点的坐标 (x,y)
- ◆ draw_points——画多个点，参数：点的坐标列表，x,y都是列表

绘制调试信息

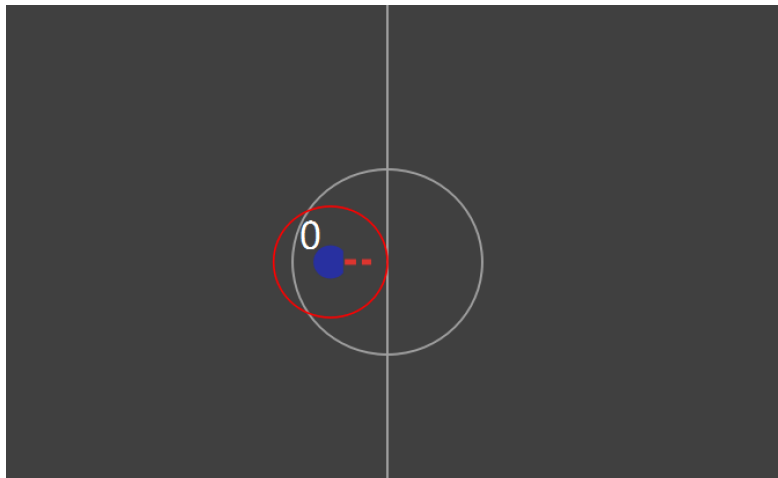
注意每次发送新的调试信息，会将上次发送的覆盖掉

```
from debug import Debugger
from zss_debug_pb2 import Debug_Msgs

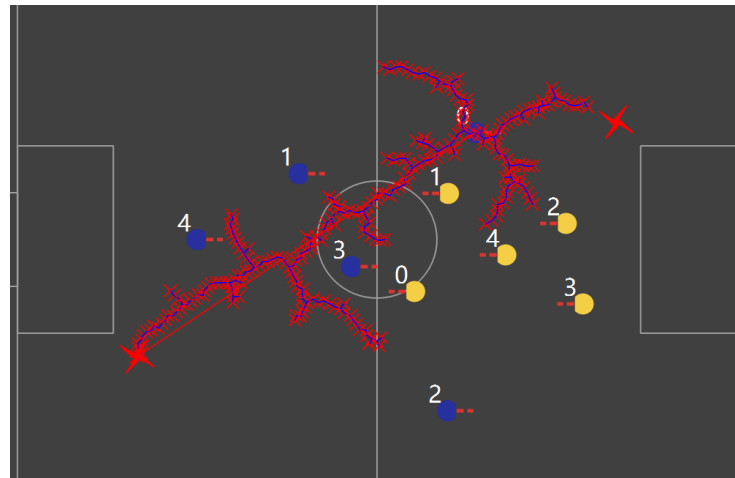
# 实例化Debugger这个类
debugger = Debugger()
# 定义一个调试信息的包
package = Debug_Msgs()
# 画一个圆
debugger.draw_circle(package, x=0, y=500)
# 画一条线
debugger.draw_line(package, x1=0, y1=2500, x2=600, y2=2500)
# 画多条线
debugger.draw_lines(package, x1=[0,0], y1=[0,2000], x2=[2000,2000], y2=[0,2000])
# 画一个点
debugger.draw_point(package, x=500, y=500)
# 画多个点
debugger.draw_points(package, x=[1000, 2000], y=[3000, 3000])
# 发送调试信息
debugger.send()
```

绘制调试信息

Example



绘制一个半径30cm的圆



绘制拓展的RRT树

主程序

```
from vision import Vision
from action import Action
from debug import Debugger
from zss_debug_pb2 import Debug_Msgs
import time

if __name__ == '__main__':
    vision = Vision()
    action = Action()
    debugger = Debugger()
    while True:
        # 1. path planning & velocity planning
        # Do something

        # 2. send command
        action.sendCommand(vx=100, vy=0, vw=0)

        # 3. draw debug msg
        package = Debug_Msgs()
        debugger.draw_circle(package, vision.my_robot.x, vision.my_robot.y)
        debugger.send(package)

    time.sleep(0.01)
```

在main.py中实现了主函数，完成了图像数据的接收、控制指令发送、调试信息绘制。

- ◆ 图像数据接收：通过实例化Vision这个类完成，会单独开一个线程读取图像数据，只需访问它的成员变量即可获得场上机器人的位置等信息。

主程序

```
from vision import Vision
from action import Action
from debug import Debugger
from zss_debug_pb2 import Debug_Msgs
import time

if __name__ == '__main__':
    vision = Vision()
    action = Action()
    debugger = Debugger()
    while True:
        # 1. path planning & velocity planning
        # Do something

        # 2. send command
        action.sendCommand(vx=100, vy=0, vw=0)

        # 3. draw debug msg
        package = Debug_Msgs()
        debugger.draw_circle(package, vision.my_robot.x, vision.my_robot.y)
        debugger.send(package)

    time.sleep(0.01)
```

在main.py中实现了主函数，完成了图像数据的接收、控制指令发送、调试信息绘制。

- ◆ 路径规划&速度规划：需要在注释处完成，可以实现A*或者RRT。

主程序

```
from vision import Vision
from action import Action
from debug import Debugger
from zss_debug_pb2 import Debug_Msgs
import time

if __name__ == '__main__':
    vision = Vision()
    action = Action()
    debugger = Debugger()
    while True:
        # 1. path planning & velocity planning
        # Do something

        # 2. send command
        action.sendCommand(vx=100, vy=0, vw=0)

        # 3. draw debug msg
        package = Debug_Msgs()
        debugger.draw_circle(package, vision.my_robot.x, vision.my_robot.y)
        debugger.send(package)

        time.sleep(0.01)
```

在main.py中实现了主函数，完成了图像数据的接收、控制指令发送、调试信息绘制。

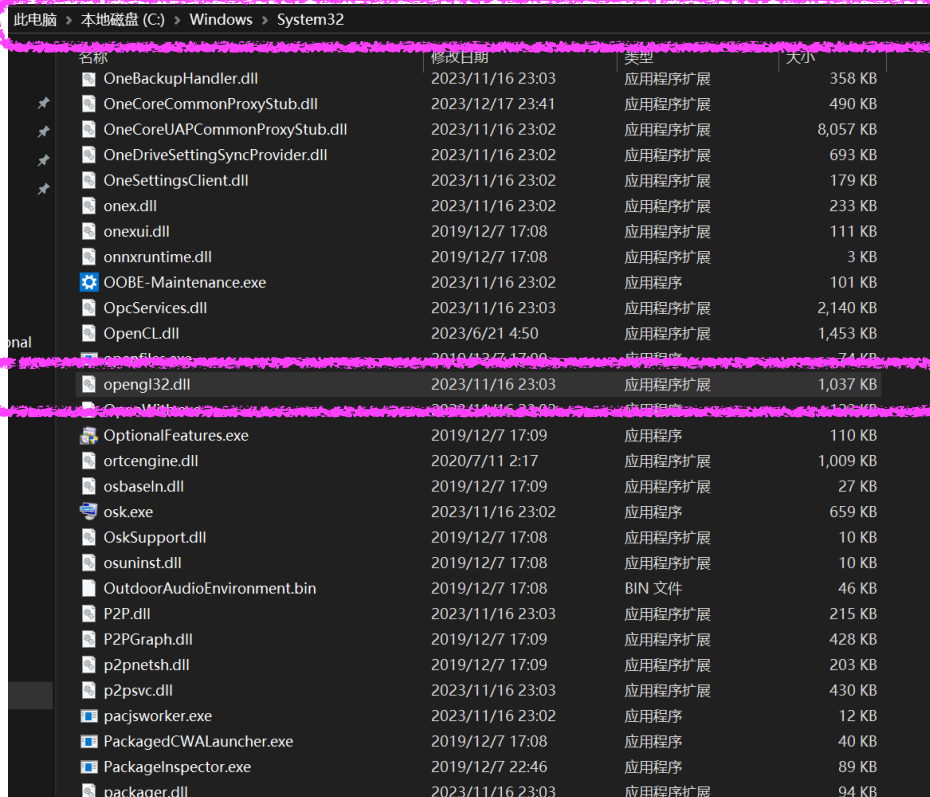
- ◆ 发送控制指令：完成规划后得到规划的速度 v_x 、 v_y ，通过Action这个类完成发送。

常见问题

Client运行白屏问题

解决方式:

1. 在系统C盘右图所示路径找到 opengl32.dll 复制;
2. 粘贴覆盖掉Client安装路径下的对应文件;



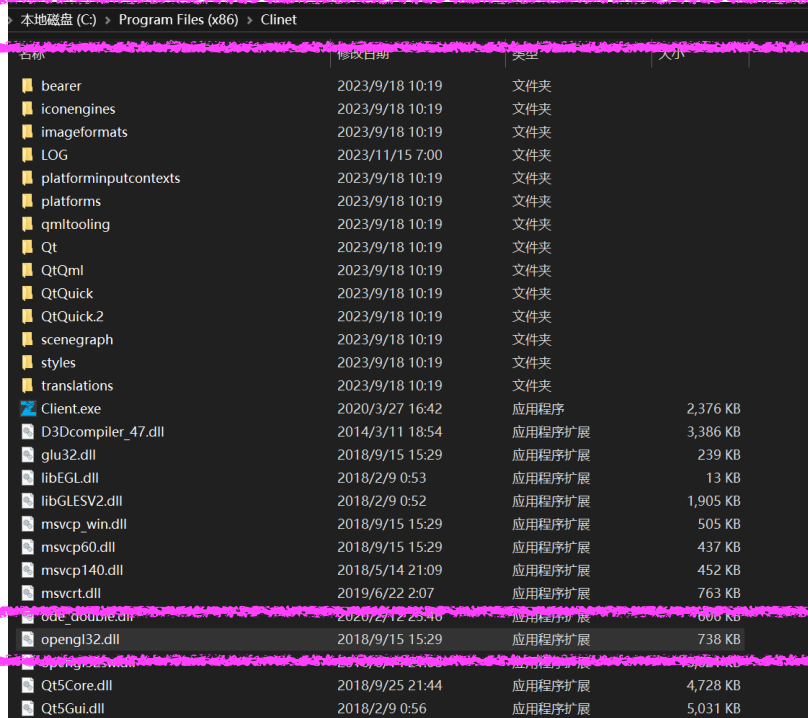
常见问题

Client运行白屏问题

解决方式：

1. 在系统C盘右图所示路径找到 opengl32.dll 复制；
2. 粘贴覆盖掉Client安装路径下的对应文件；

注：右图为实例，以实际Client安装路径为准；



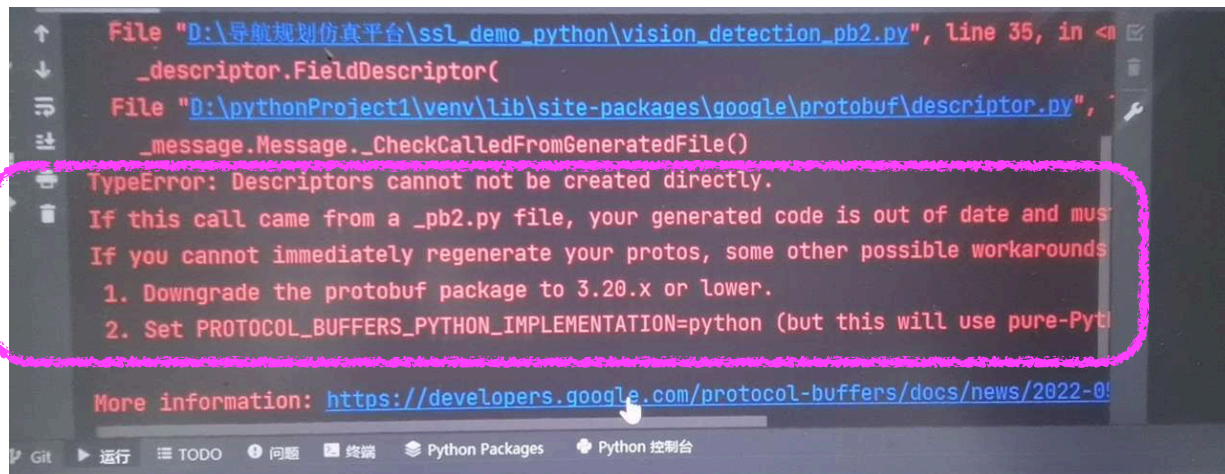
名称	修改日期	类型	大小
bearer	2023/9/18 10:19	文件夹	
iconengines	2023/9/18 10:19	文件夹	
imageformats	2023/9/18 10:19	文件夹	
LOG	2023/11/15 7:00	文件夹	
platforminputcontexts	2023/9/18 10:19	文件夹	
platforms	2023/9/18 10:19	文件夹	
qmltooling	2023/9/18 10:19	文件夹	
Qt	2023/9/18 10:19	文件夹	
QtQml	2023/9/18 10:19	文件夹	
QtQuick	2023/9/18 10:19	文件夹	
QtQuick2	2023/9/18 10:19	文件夹	
scenegraph	2023/9/18 10:19	文件夹	
styles	2023/9/18 10:19	文件夹	
translations	2023/9/18 10:19	文件夹	
Client.exe	2020/3/27 16:42	应用程序	2,376 KB
D3Dcompiler_47.dll	2014/3/11 18:54	应用程序扩展	3,386 KB
glu32.dll	2018/9/15 15:29	应用程序扩展	239 KB
libEGL.dll	2018/2/9 0:53	应用程序扩展	13 KB
libGLSV2.dll	2018/2/9 0:52	应用程序扩展	1,905 KB
msvc_p_win.dll	2018/9/15 15:29	应用程序扩展	505 KB
msvc_p60.dll	2018/9/15 15:29	应用程序扩展	437 KB
msvc_p140.dll	2018/5/14 21:09	应用程序扩展	452 KB
msvcrt.dll	2019/6/22 2:07	应用程序扩展	763 KB
code_double.dll	2020/2/12 23:40	应用程序扩展	606 KB
opengl32.dll	2018/9/15 15:29	应用程序扩展	738 KB
opengl32sw.dll	2018/9/15 15:29	应用程序扩展	1,571 KB
Qt5Core.dll	2018/9/25 21:44	应用程序扩展	4,728 KB
Qt5Gui.dll	2018/2/9 0:56	应用程序扩展	5,031 KB

常见问题

protobuf版本问题

解决方式：

1. 使用助教发到钉钉群的代码，以使用新版本的protobuf；
2. 或者将protobuf回退到老版本；



The screenshot shows a Python IDE with a dark theme. A red error message is displayed in the console, which is highlighted by a pink dashed border. The message reads: "TypeError: Descriptors cannot not be created directly. If this call came from a _pb2.py file, your generated code is out of date and must be regenerated. Running proto_compile will regenerate your proto files. If you cannot immediately regenerate your protos, some other possible workarounds: 1. Downgrade the protobuf package to 3.20.x or lower. 2. Set PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python (but this will use pure-Python implementations and is much slower than C). More information: https://developers.google.com/protocol-buffers/docs/news/2022-08-01". The IDE interface includes a sidebar on the left with icons for Explorer, Search, and Run and Debug. The bottom status bar shows "Git", "运行", "TODO", "问题", "终端", "Python Packages", and "Python 控制台".

```
File "D:\导航规划仿真平台\ssl_demo_python\vision_detection_pb2.py", line 35, in <module>
    _descriptor.FieldDescriptor(
File "D:\pythonProject1\venv\lib\site-packages\google\protobuf\descriptor.py", line 111, in __new__
    _message.Message._CheckCalledFromGeneratedFile()
TypeError: Descriptors cannot not be created directly.
If this call came from a _pb2.py file, your generated code is out of date and must be regenerated.
Running proto_compile will regenerate your proto files.
If you cannot immediately regenerate your protos, some other possible workarounds:
  1. Downgrade the protobuf package to 3.20.x or lower.
  2. Set PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python (but this will use pure-Python implementations
and is much slower than C).

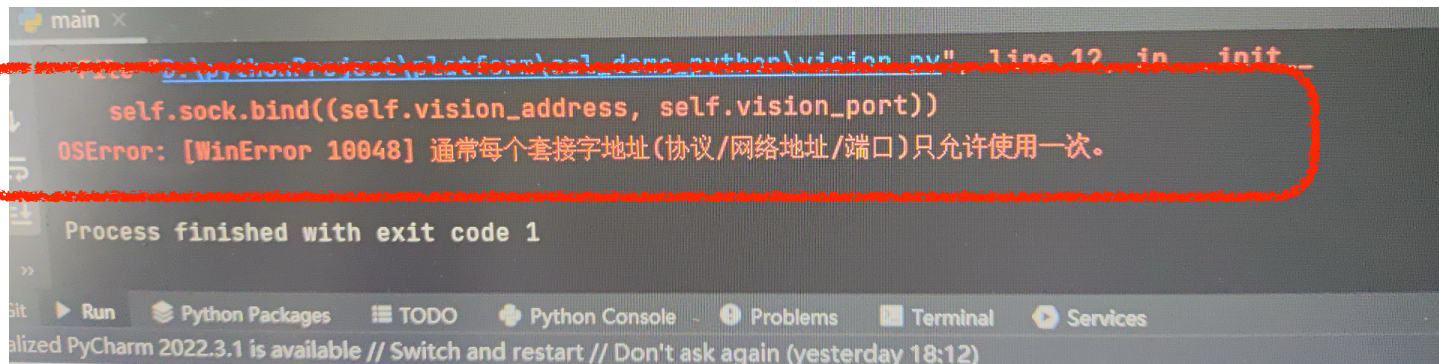
More information: https://developers.google.com/protocol-buffers/docs/news/2022-08-01
```


常见问题

端口占用问题

解决方式：

1. 打开Client后，点击快速关闭按钮，将其彻底关闭，防止后台运行占用端口；
2. 避免代码同时运行多次，保证 Vison、Action和Debugger类只有一个实例；



常见问题

无法收到视觉反馈

解决方式：在创建 Vision 类后，sleep一小段时间再进入while循环即可。

```
mo python/vision.py
My robot: -999999 -999999
Blue robot 0: -999999 -999999
Yellow robot 0: -999999 -999999
```

```
6
7 if __name__ == '__main__':
8     vision = Vision()
9     action = Action()
10    debugger = Debugger()
11    time.sleep(0.1)
12    while True:
13        # 1. path planning & velocity planning
14        # Do something
15
16        # 2. send command
17        action.sendCommand(vx=100, vy=0, vw=0)
18
19        # 3. draw debug msg
20        package = Debug_Msgs()
21        debugger.draw_circle(package, vision.my_robot.x, vision.my_robot.y)
22        debugger.send(package)
23
24        time.sleep(0.01)
25
```



谢谢大家
